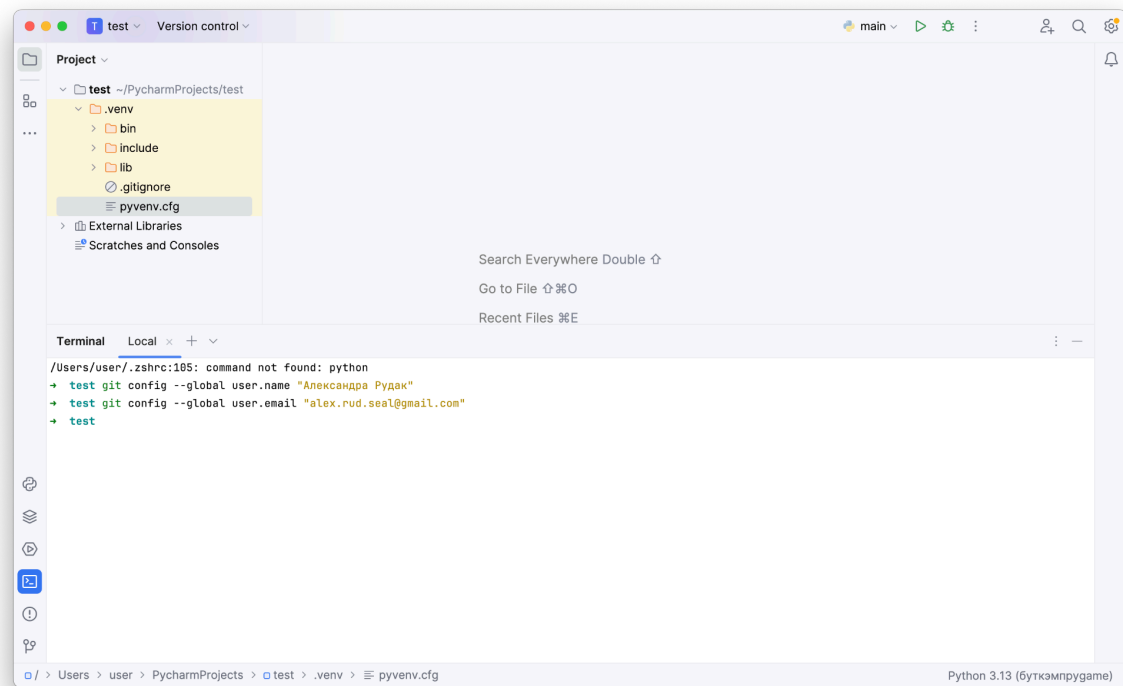


Шаг 0. Установка и настройка окружения

Перед началом работы необходимо убедиться, что Git установлен и настроен.

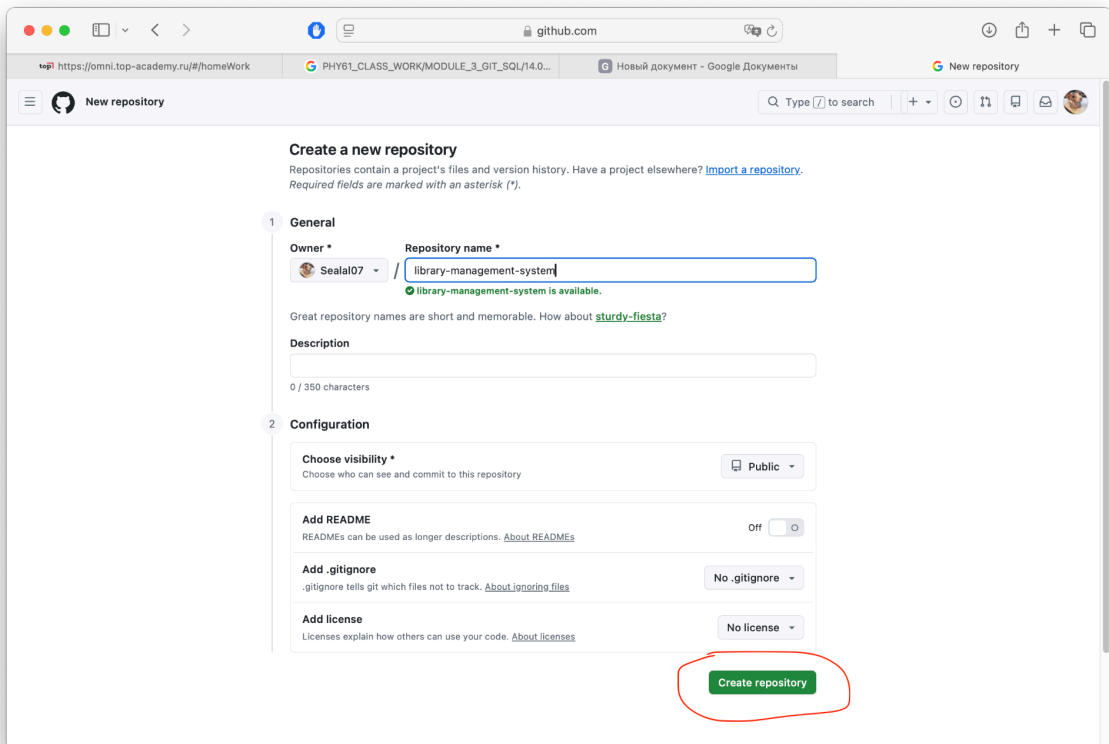
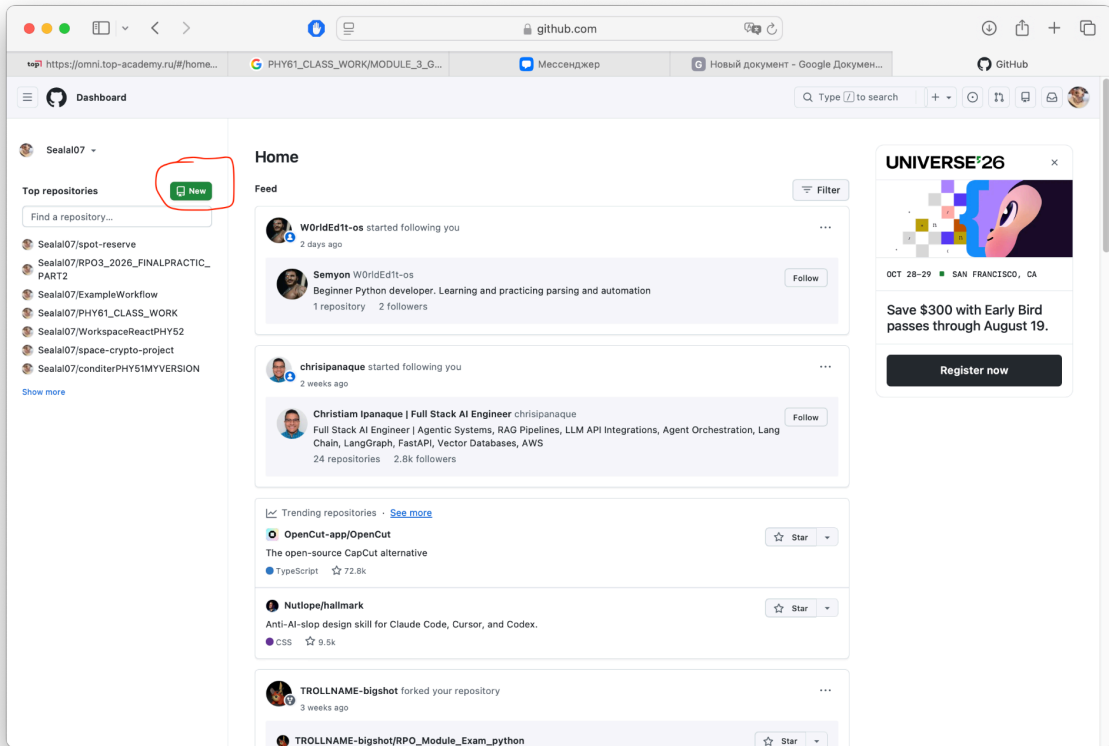
1. Откройте терминал
2. Выполните команды настройки:

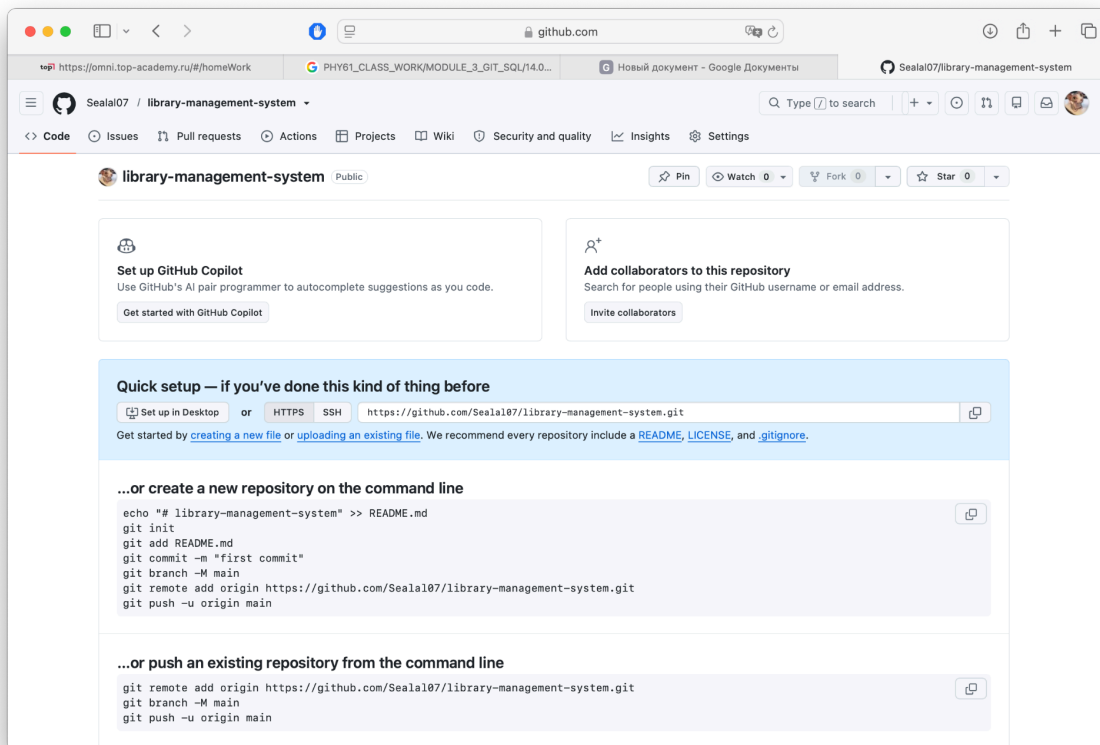
```
git config --global user.name "Ваше Имя"  
git config --global user.email "ваш@email.com"
```



Шаг 1. Создание репозитория на GitHub

1. Зайдите в свой аккаунт на [GitHub](#).
2. В правом верхнем углу нажмите на плюсик **+** и выберите **New repository**.
3. Заполните поля:
 - **Repository name:** `library-management-system`
 - **Public.**
 - **Важно!** Не ставьте галочки у *Add a README file*, *Add .gitignore* и *Choose a license*. Репозиторий должен быть абсолютно пустым.
4. Нажмите кнопку **Create repository**.

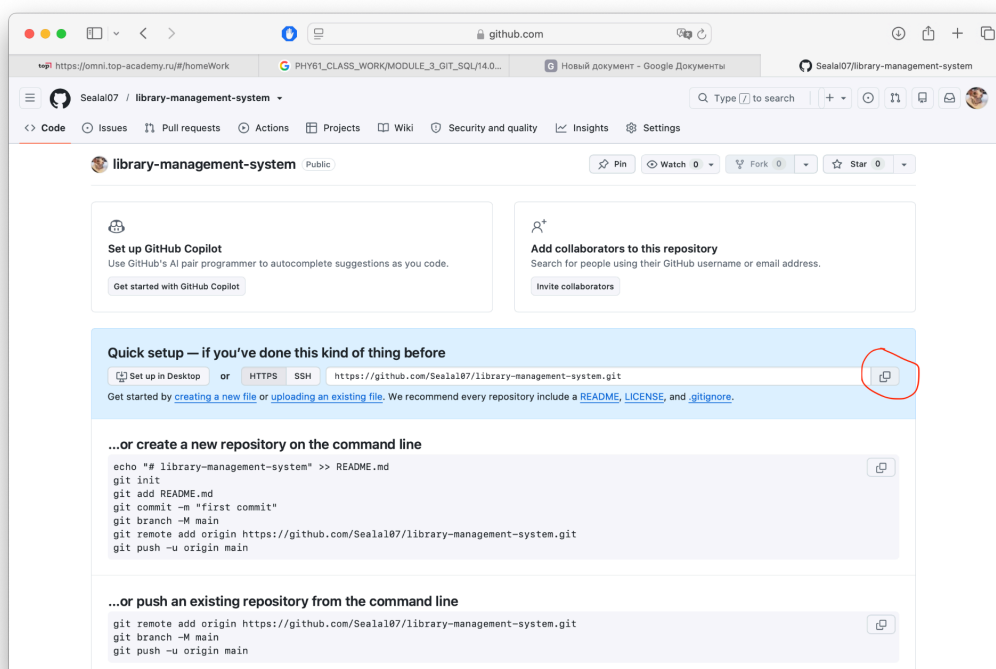


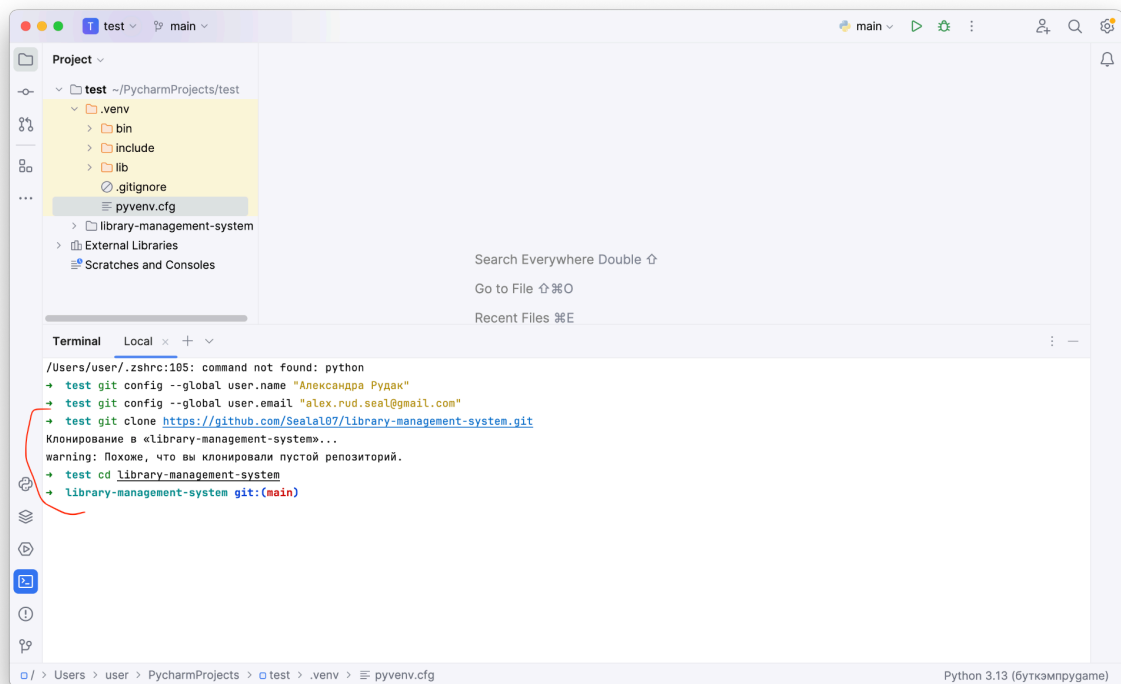


Шаг 2. Клонирование репозитория на ПК

1. Скопируйте ссылку на ваш репозиторий (кнопка копирования рядом с HTTPS-адресом).
2. Откройте терминал на компьютере, перейдите в папку, где вы хотите хранить проект (например, `cd Documents`), и введите:

```
git clone https://github.com/ВАШ-ЛОГИН/library-management-system.git
cd library-management-system
```





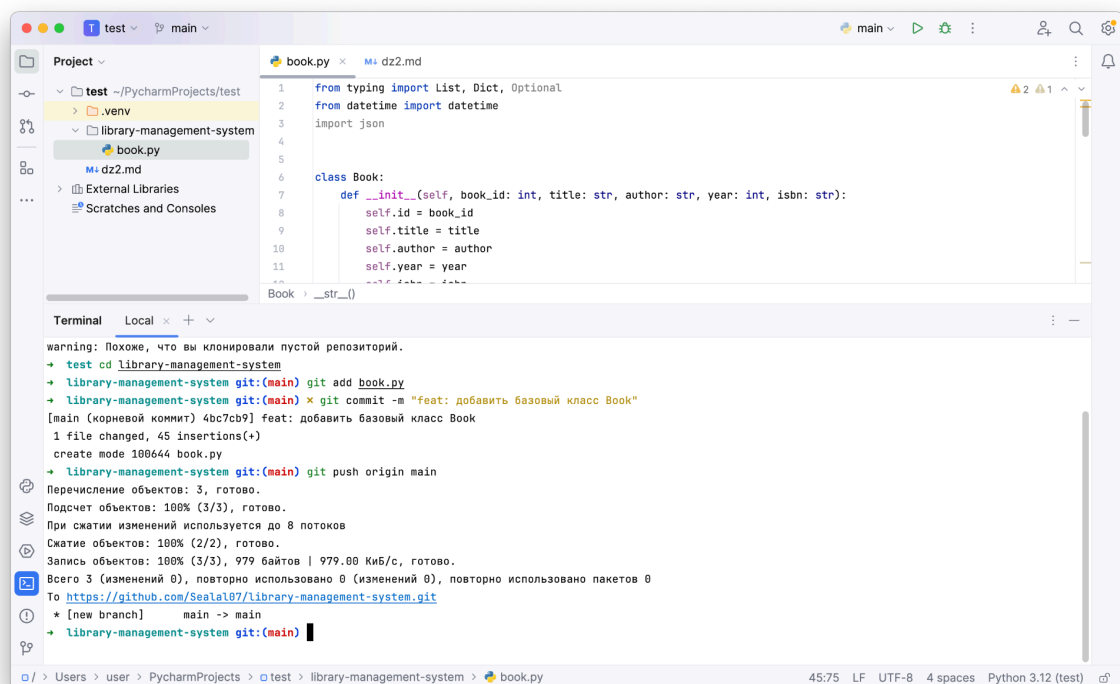
Шаг 3. Создание базового кода в ветке **main**

1. Внутри папки **library-management-system** создайте файл **book.py** с кодом из задания (класс **Book**).
2. Откройте терминал и выполните команды для отправки кода на GitHub:

```
git add book.py
```

```
git commit -m "feat: добавить базовый класс Book"
```

```
git push origin main
```



Шаг 4. Создание ветки **feature/library** и класса **Library**

1. Создаем новую ветку для разработки и переключаемся на нее:

```
git checkout -b feature/library
```

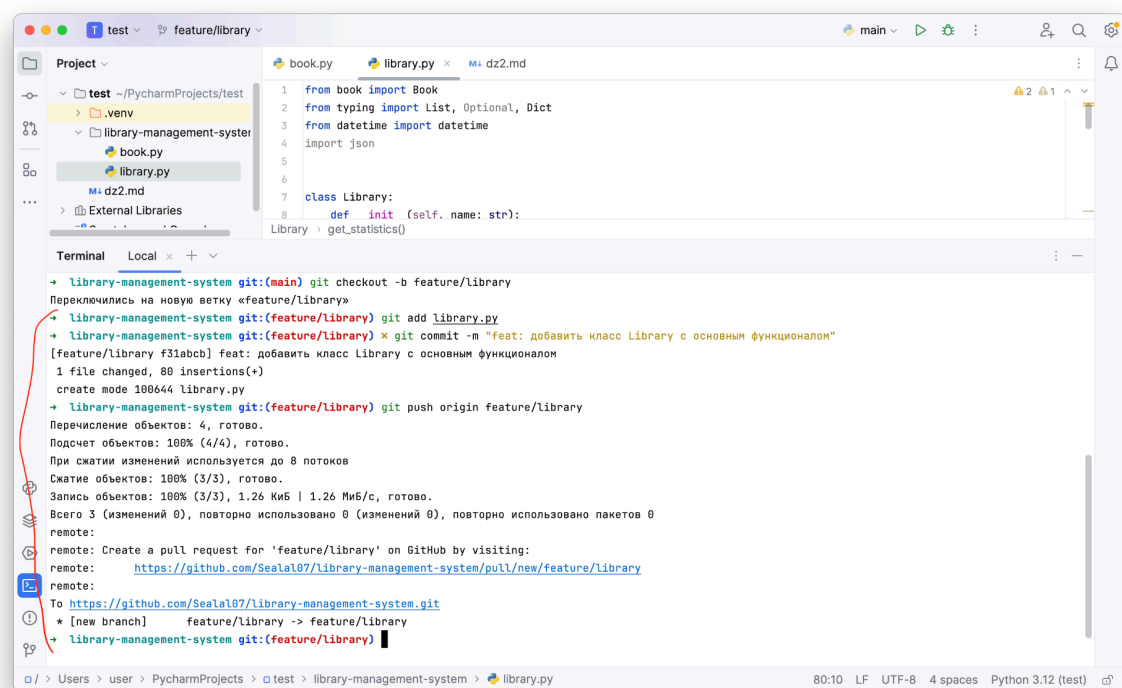


2. Создайте файл **library.py** и вставьте туда код класса **Library** из задания.
3. Добавляем изменения в индекс, коммитим и отправляем ветку на GitHub:

```
git add library.py
```

```
git commit -m "feat: добавить класс Library с основным функционалом"
```

```
git push origin feature/library
```



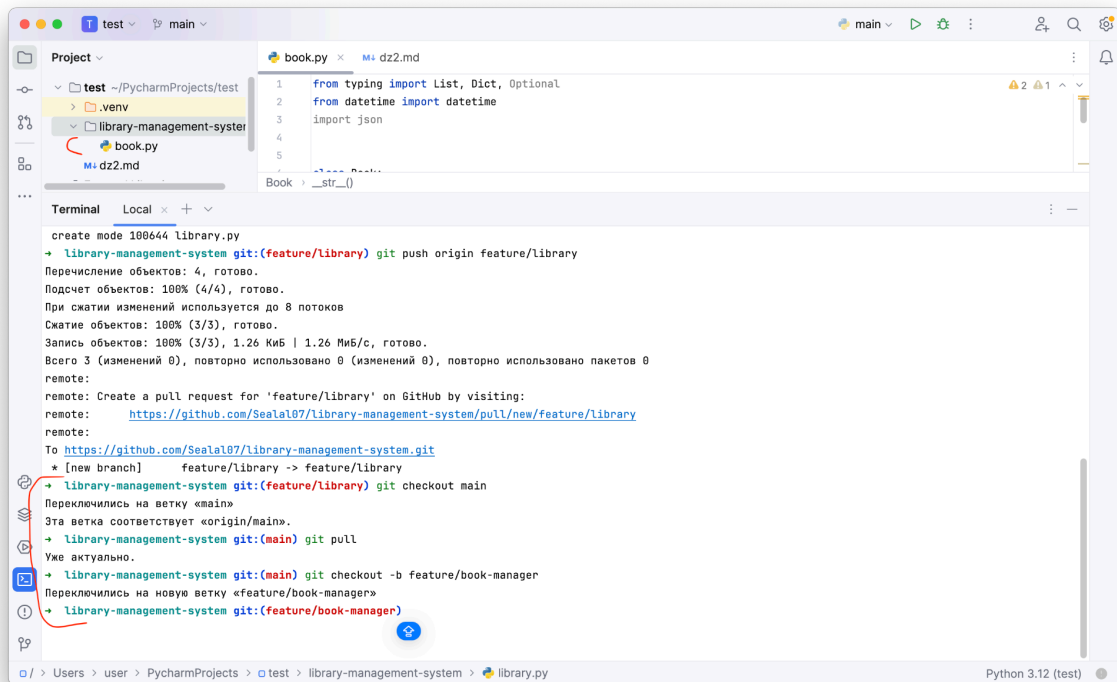
Шаг 5. Создание ветки **feature/book-manager**

1. Сначала возвращаемся в **main**, чтобы забрать свежие изменения (если они были), а затем создаем вторую ветку разработки:

```
git checkout main
```

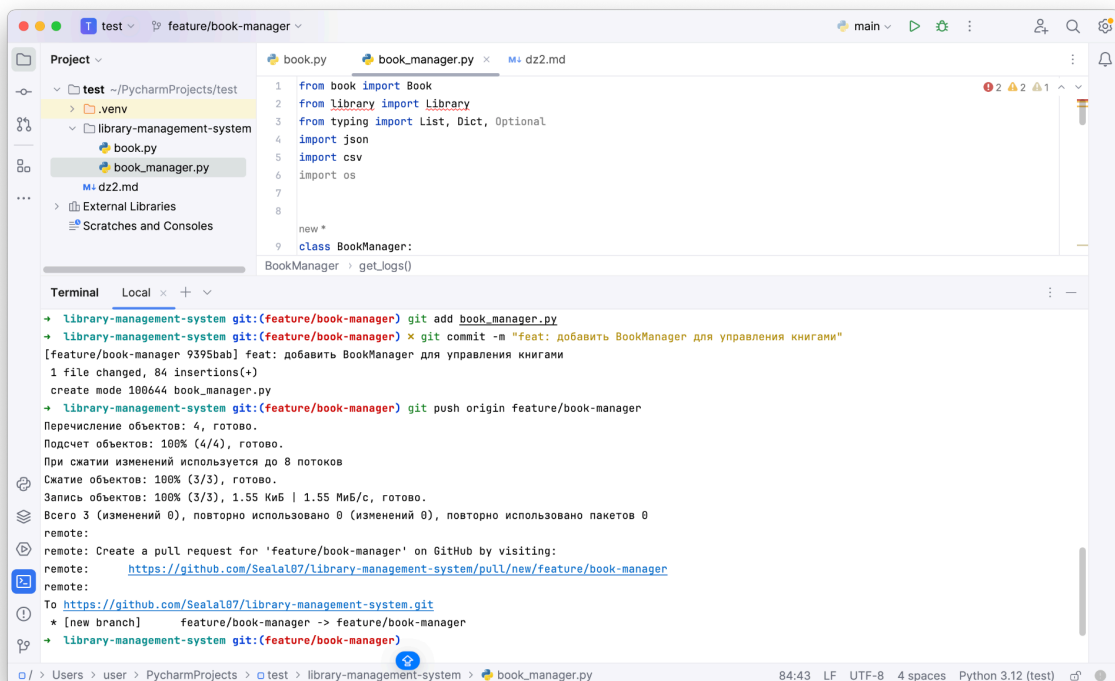
```
git pull
```

```
git checkout -b feature/book-manager
```



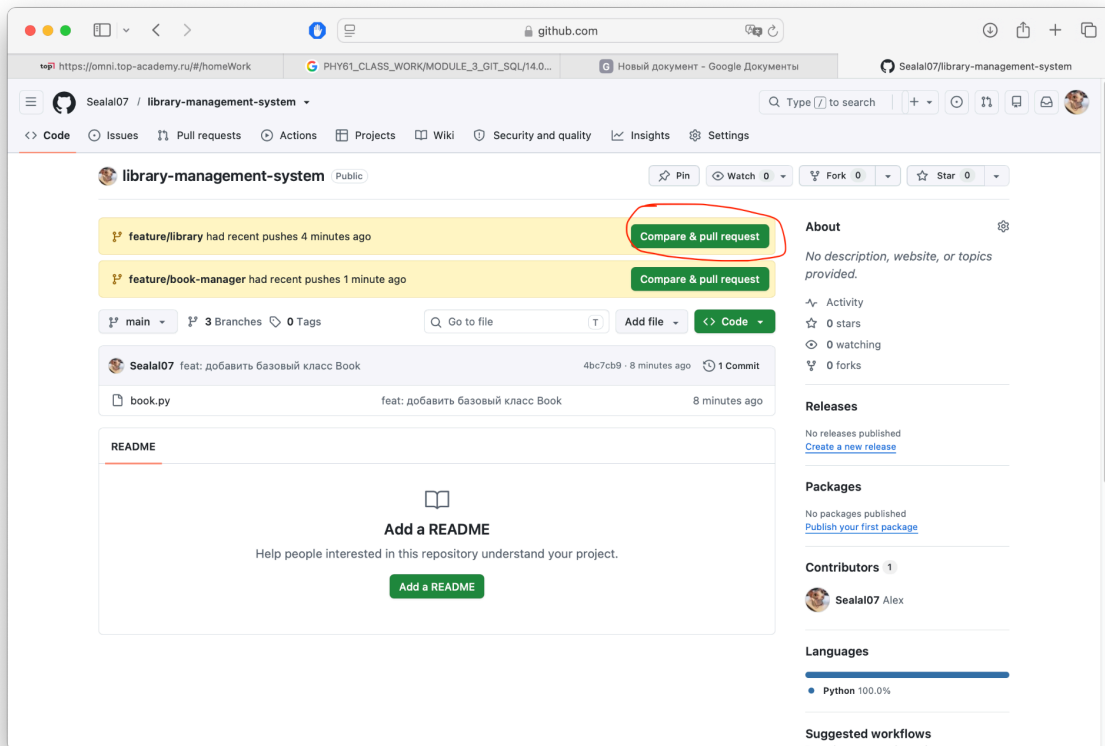
2. Создайте файл **book_manager.py** и скопируйте в него код класса **BookManager** из задания.
3. Отправляем ветку в облако:

```
git add book_manager.py
git commit -m "feat: добавить BookManager для управления книгами"
git push origin feature/book-manager
```

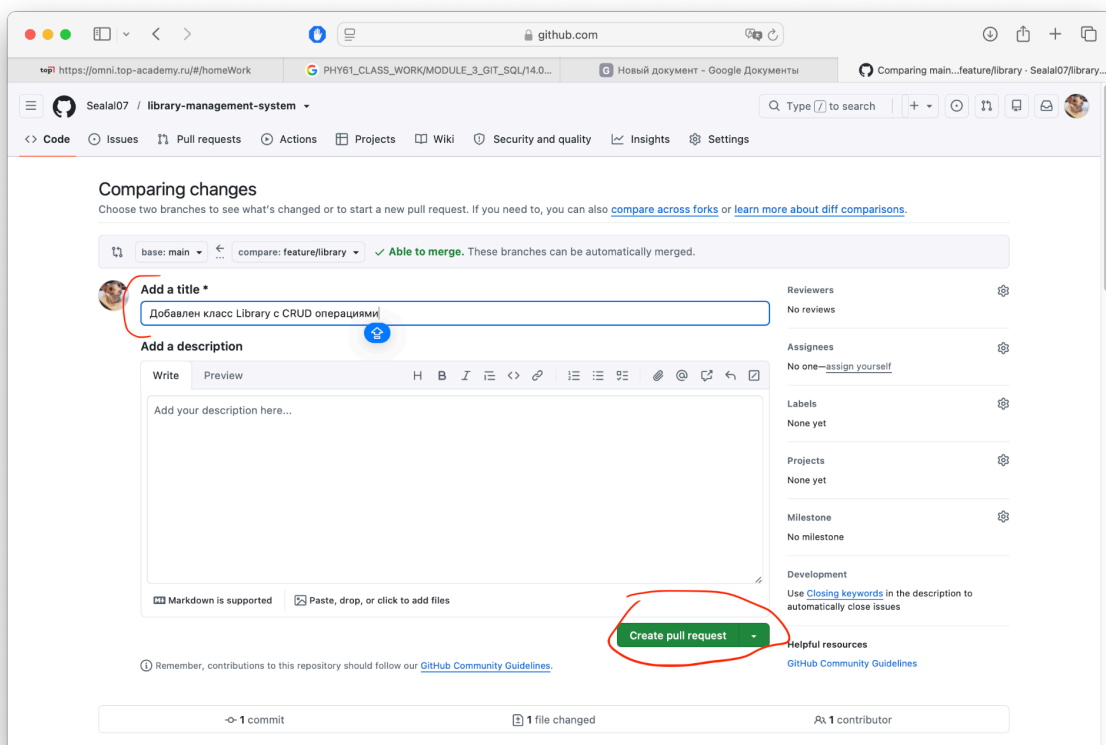


Шаг 6. Слияние ветки **feature/library**

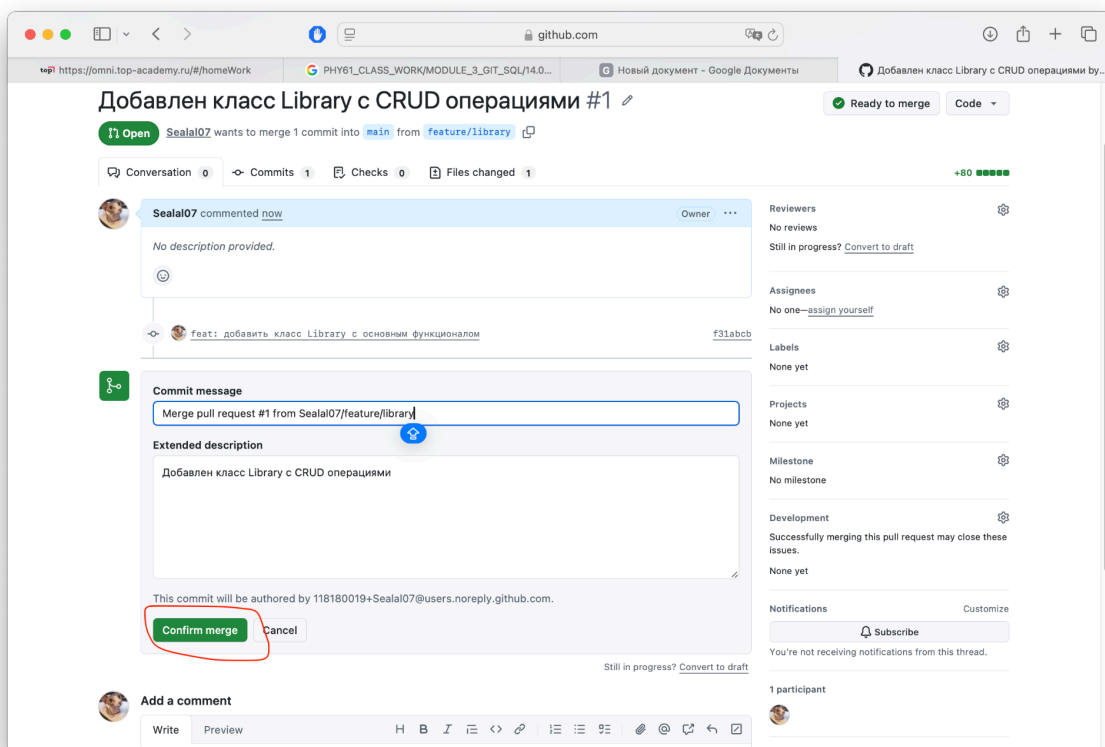
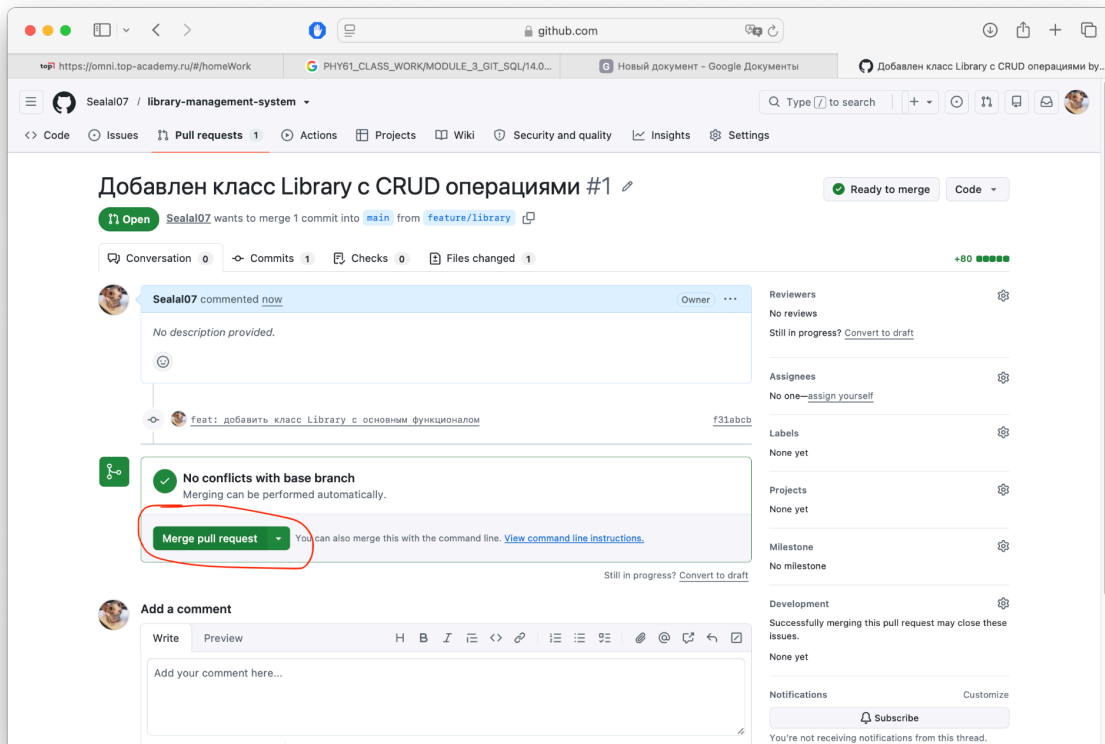
1. Зайдите на свой репозиторий на GitHub. Вы увидите желтую плашку с предложением сделать Pull Request.
2. Нажмите **Compare & pull request** напротив ветки **feature/library**.



3. Укажите заголовок, добавьте в описание: *"Добавлен класс Library с CRUD операциями"* и нажмите **Create pull request**.

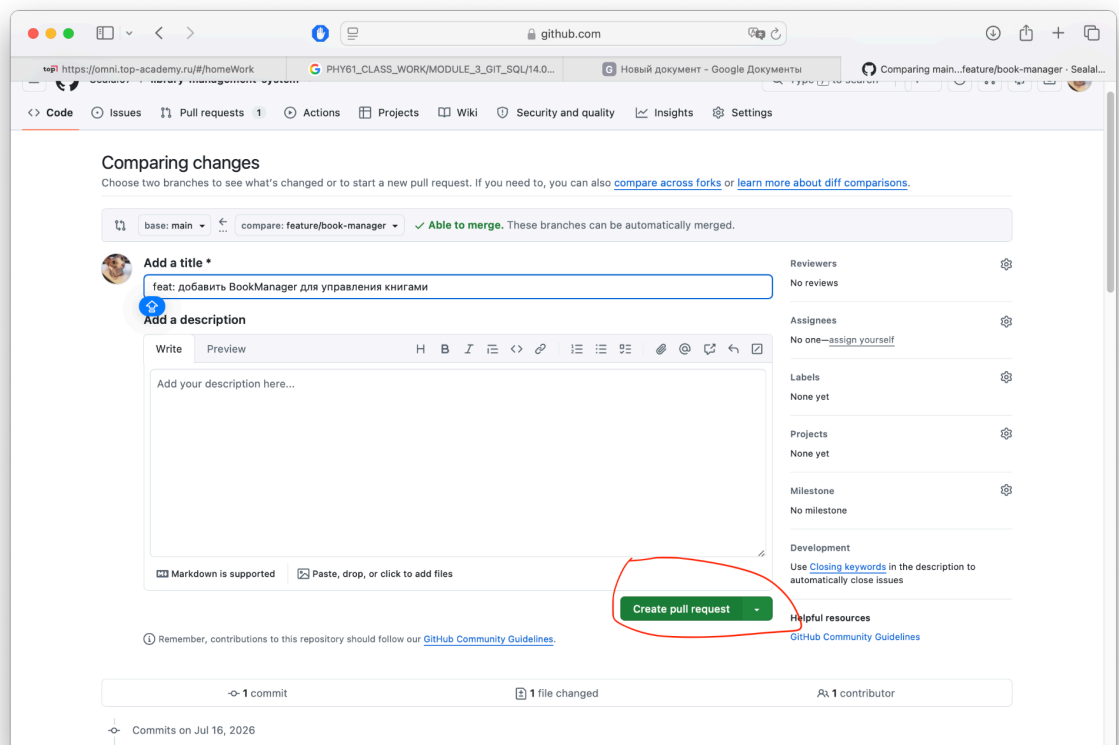
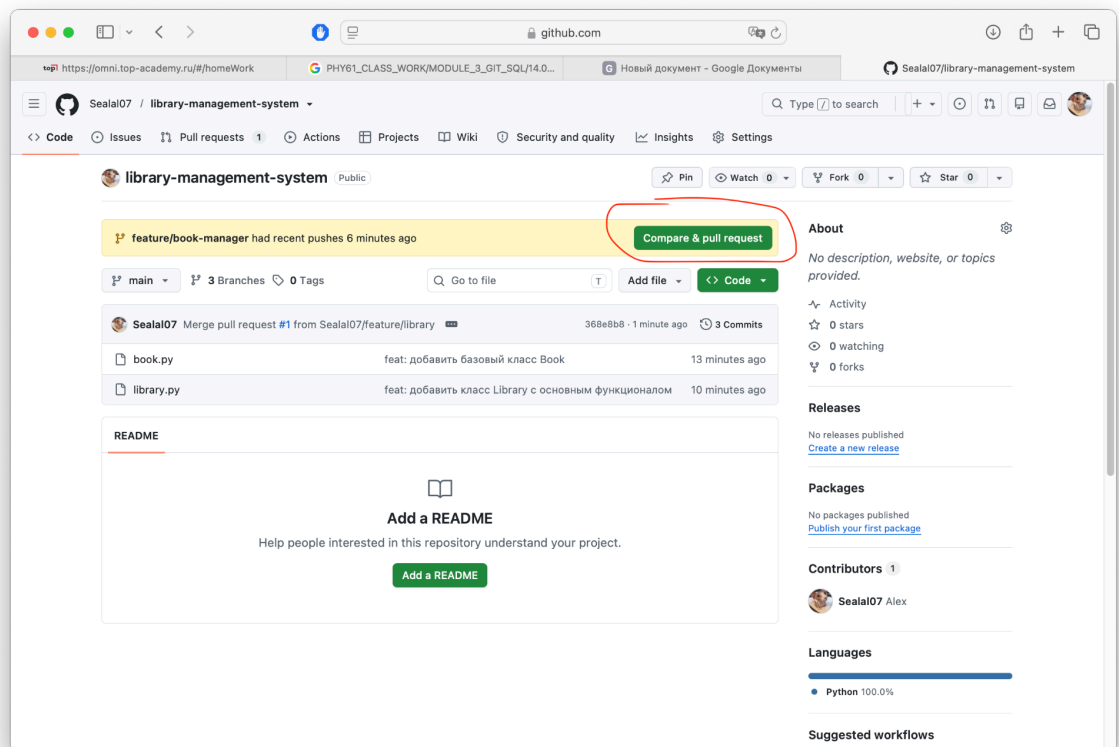


4. Так как конфликтов нет, нажмите зеленую кнопку **Merge pull request**, а затем **Confirm merge**.



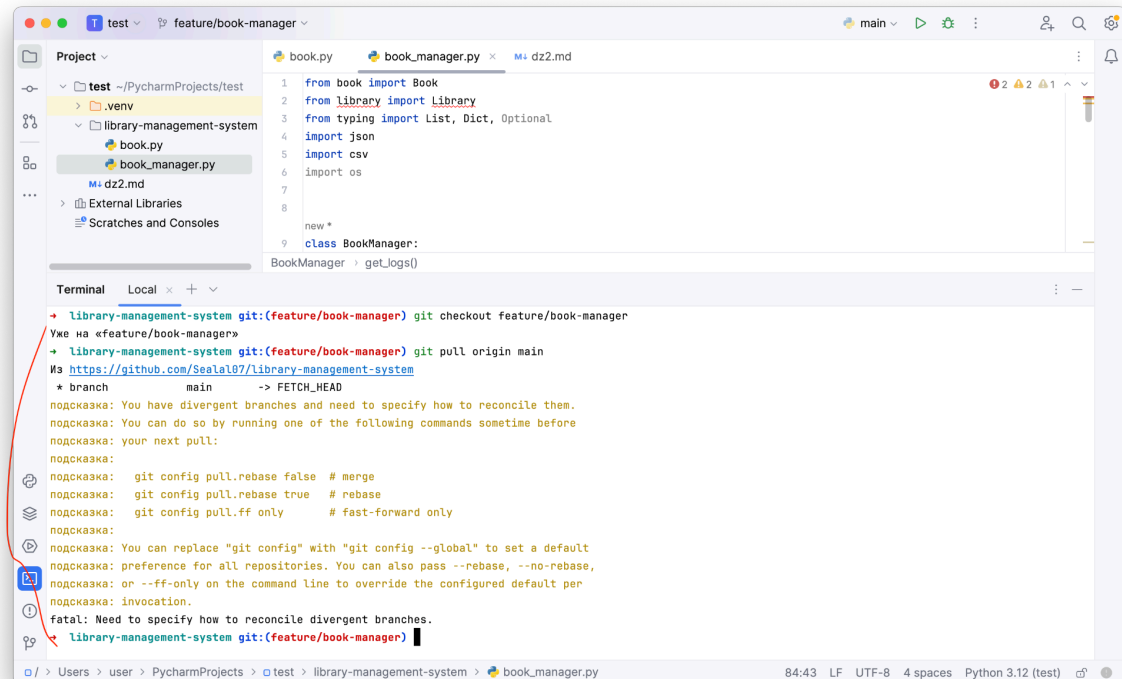
Шаг 7. Слияние ветки **feature/book-manager**

1. Создайте аналогичный Pull Request для ветки **feature/book-manager**.



2. Перед мержем обновим ветку локально, чтобы убедиться в отсутствии конфликтов:

```
git checkout feature/book-manager  
git pull origin main
```



В чем проблема?

Git просит вас один раз выбрать стратегию объединения веток по умолчанию для команды **git pull**.

Для учебных проектов и повседневной работы проще и нагляднее всего использовать стандартное слияние (**merge**).

Как это исправить (пошагово)

Шаг 1. Настраиваем Git на обычное слияние (merge)

Выполните эту команду, чтобы Git больше никогда не задавал этот вопрос и всегда делал обычный merge:

```
git config --global pull.rebase false
```

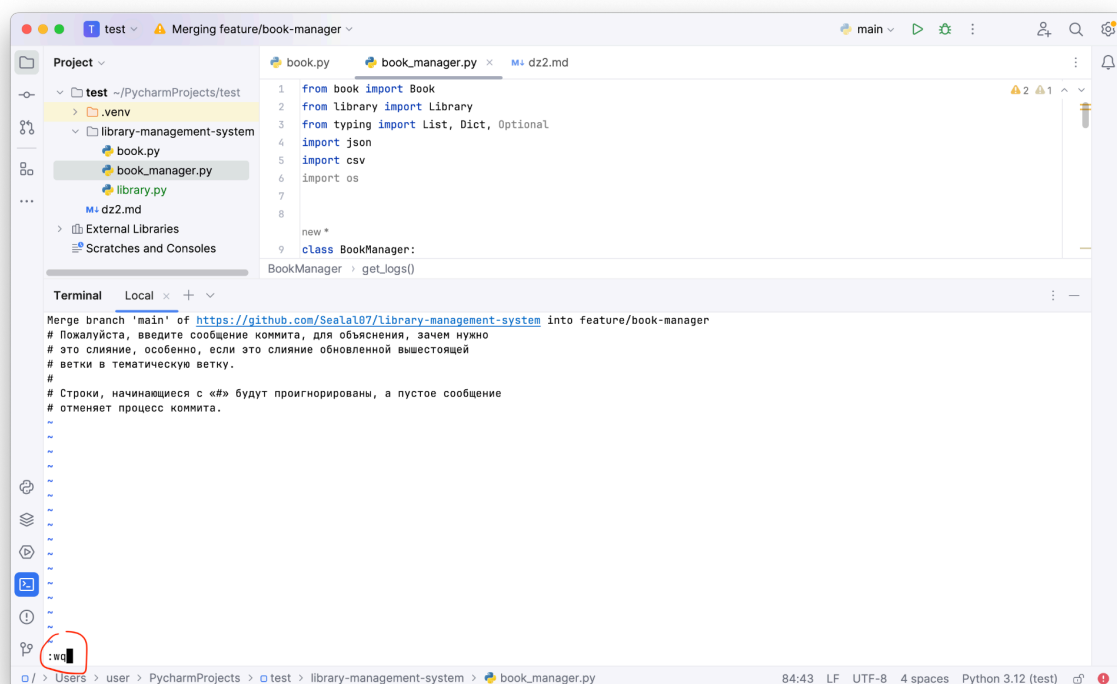
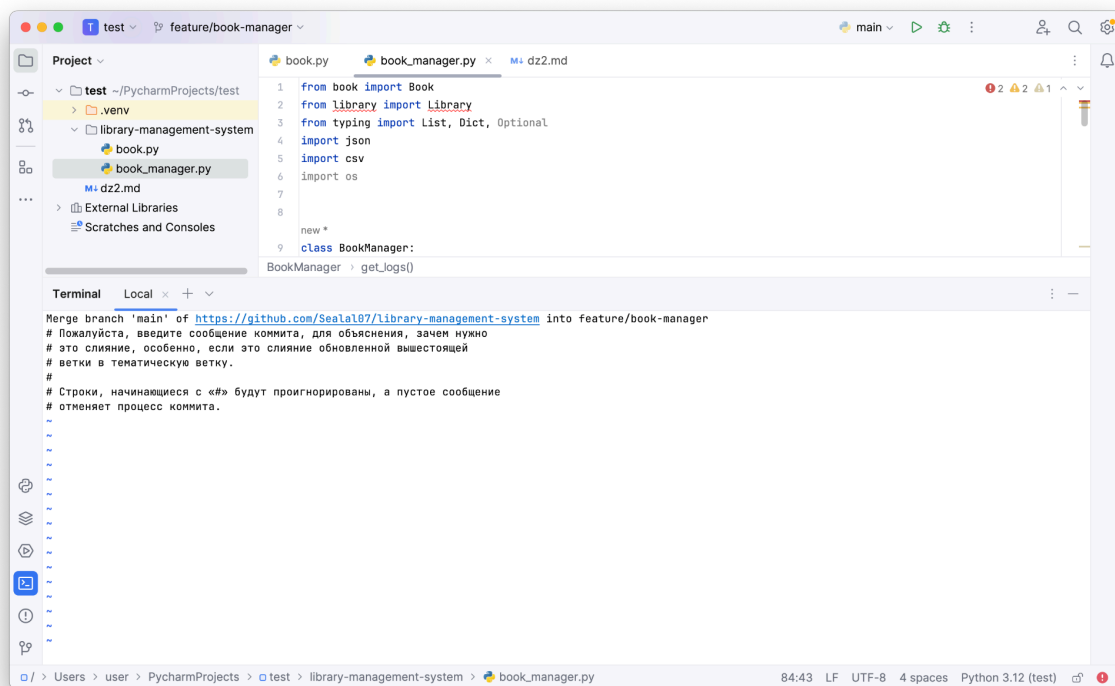
Шаг 2. Повторяем попытку стянуть изменения

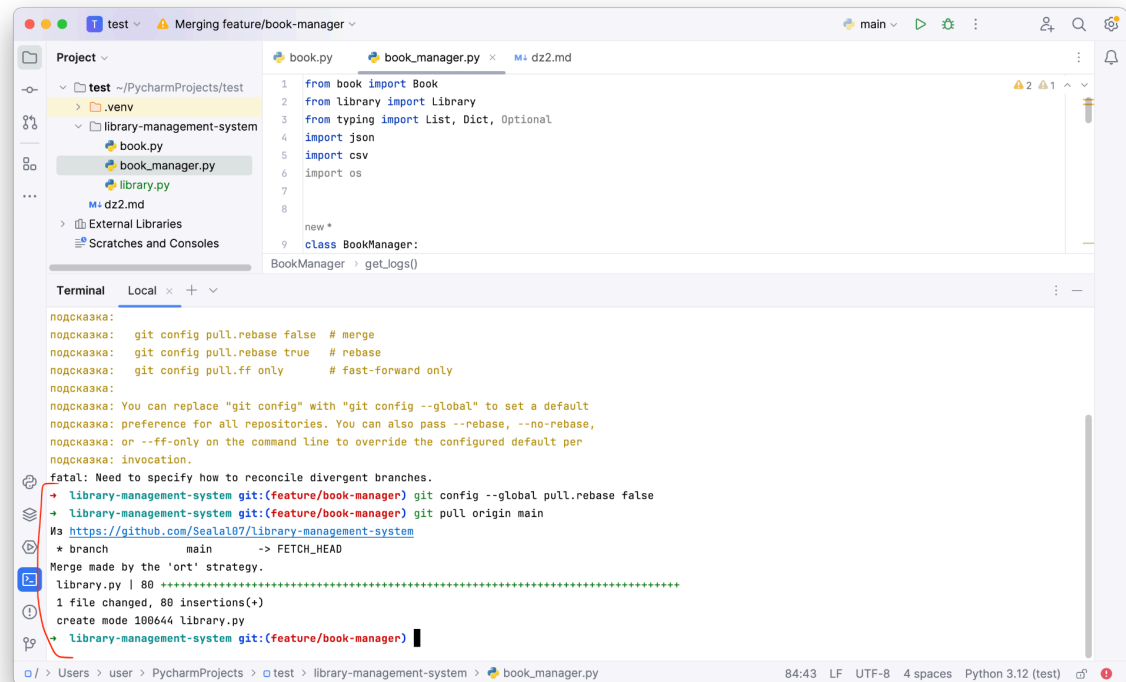
Теперь снова запускаем команду из Шага 7:

```
git pull origin main
```

Примечание: Если после этого откроется текстовый редактор в консоли (обычно это Vim или Nano) с предложением ввести текст коммита вроде **Merge branch 'main' into...**, студенту нужно просто сохранить файл и закрыть его.

- Если открылся **Vim**: набрать на клавиатуре **:wq** и нажать **Enter**.
- Если открылся **Nano**: нажать **Ctrl + O**, затем **Enter**, а затем **Ctrl + X**.

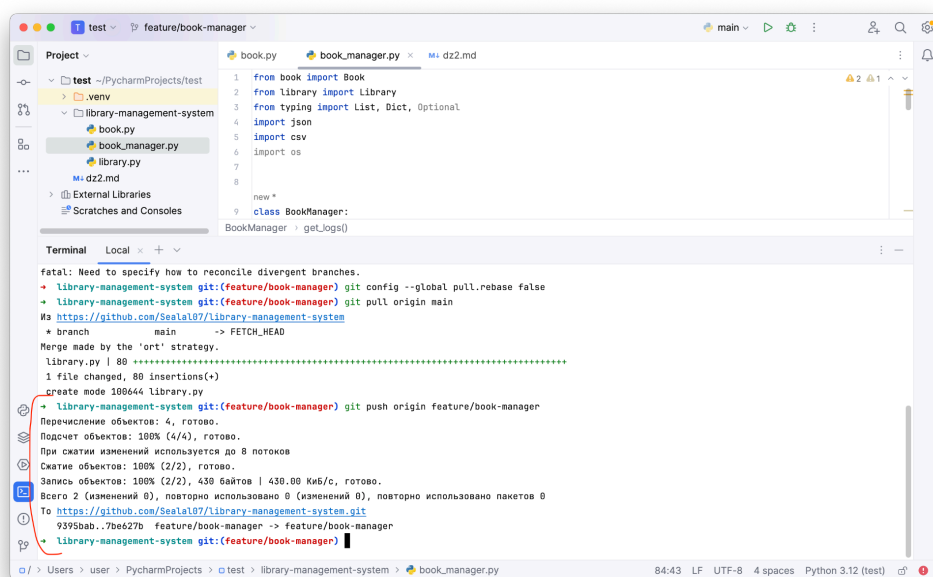




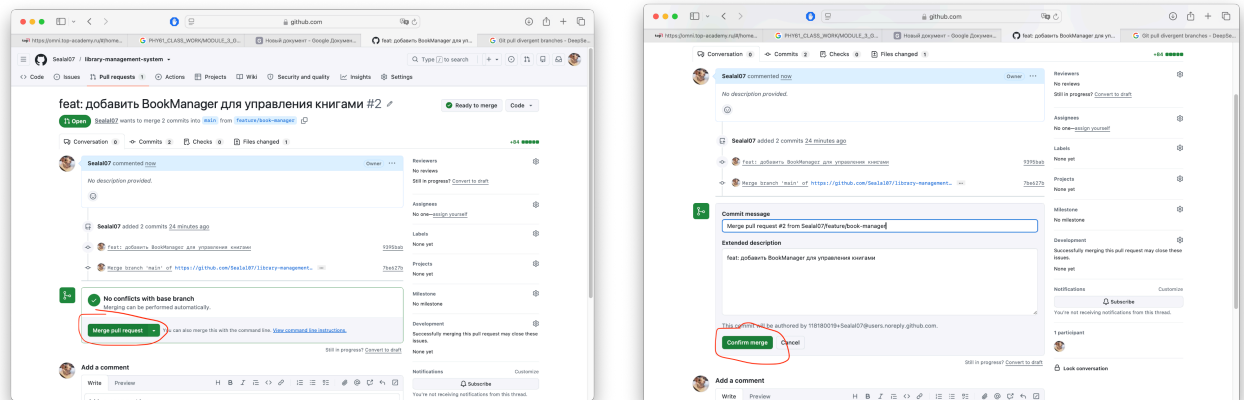
Шаг 3. Отправляем обновленную ветку на GitHub

После успешного слияния отправляем всё в облако:

```
git push origin feature/book-manager
```



3. На GitHub подтвердите слияние (Merge pull request) для этого PR.

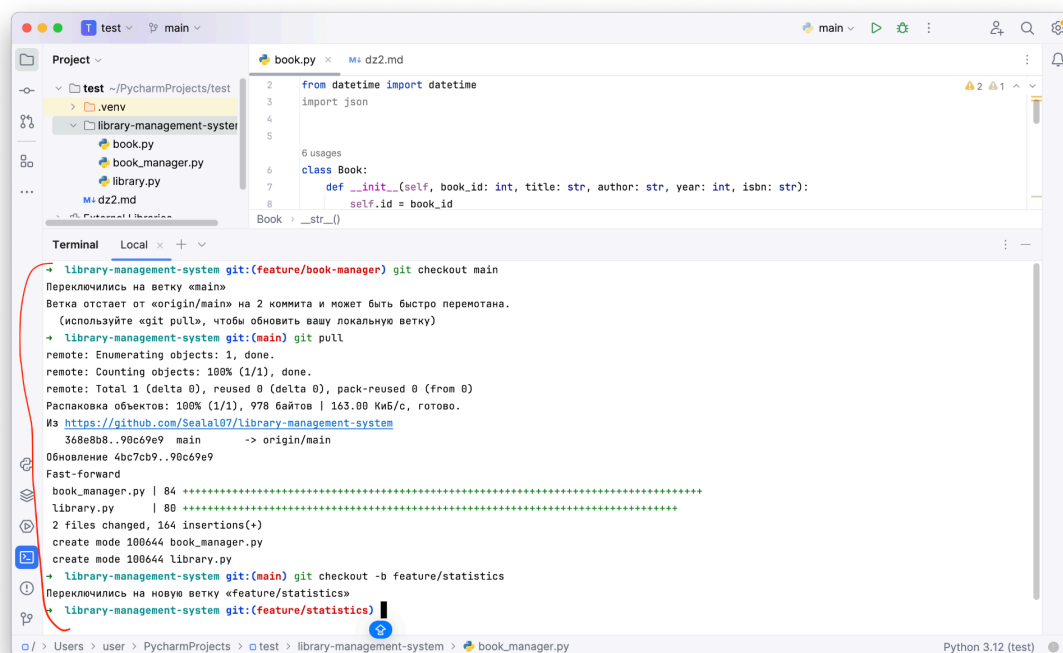


Шаг 8. Создание первой ветки для конфликта (feature/statistics)

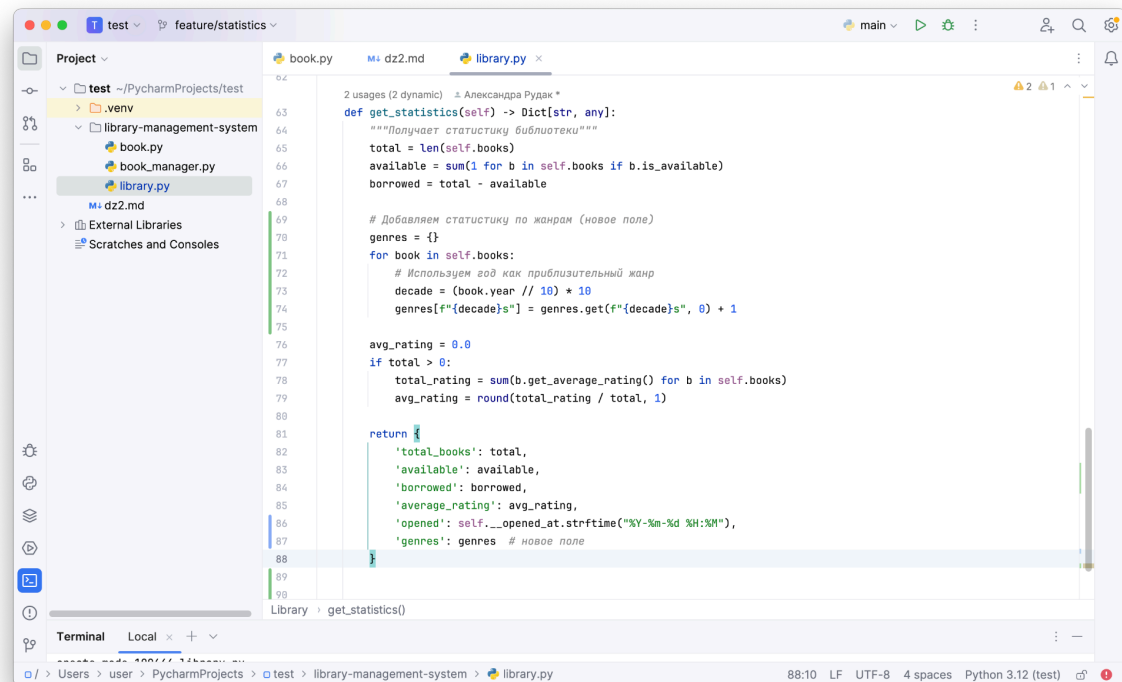
Теперь мы искусственно создадим конфликт, чтобы научиться его решать.

1. Локально перейдите в ветку **main**, стяните все изменения из GitHub и создайте ветку **feature/statistics**:

```
git checkout main
git pull
git checkout -b feature/statistics
```

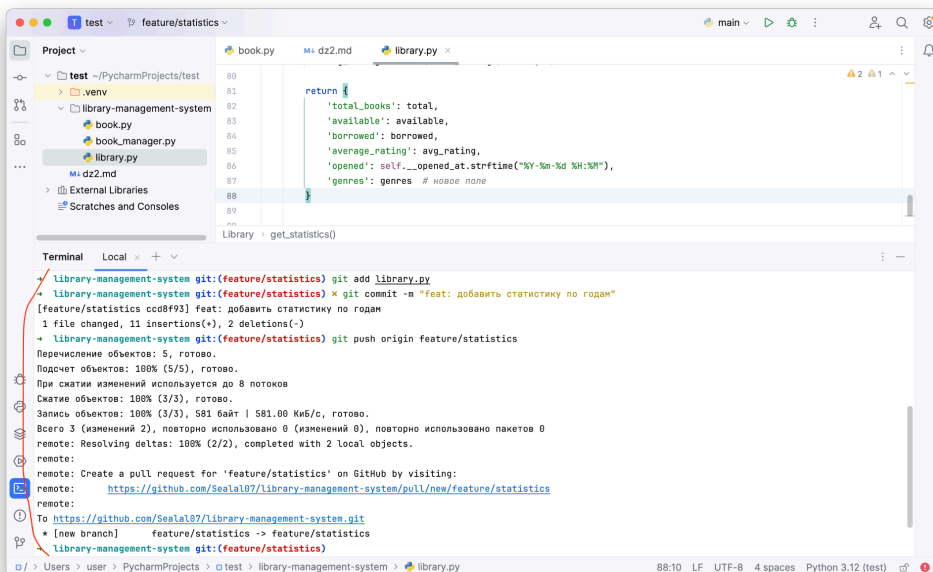


- Откройте файл `library.py` и замените метод `get_statistics` на версию с подсчетом статистики по годам (см. Шаг 8 в задании).



- Сохраните файл и отправьте код:

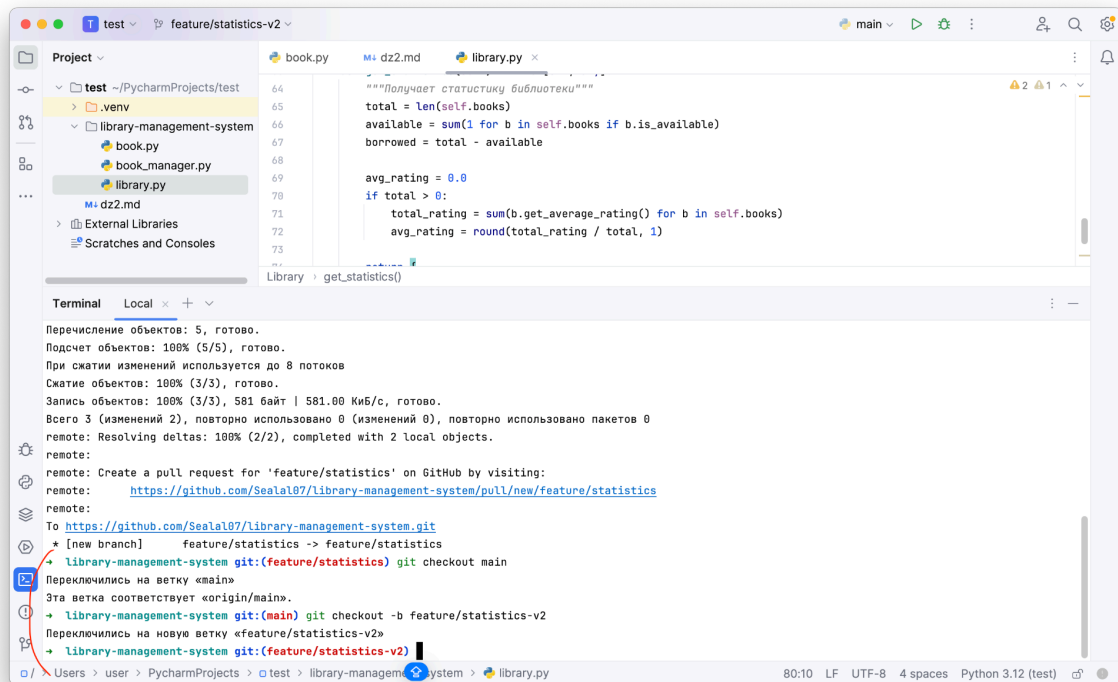
```
git add library.py
git commit -m "feat: добавить статистику по годам"
git push origin feature/statistics
```



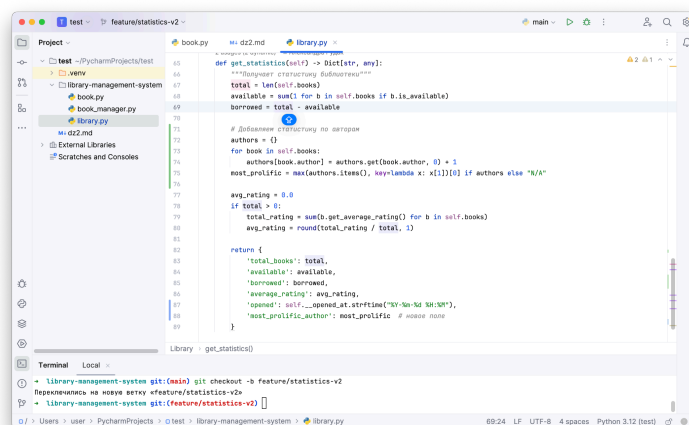
Шаг 9. Создание параллельного изменения (feature/statistics-v2)

1. Возвращаемся в `main` и создаем альтернативную ветку:

```
git checkout main
git checkout -b feature/statistics-v2
```



2. Откройте файл `library.py`. Обратите внимание: изменений из ветки `statistics` здесь нет, тут старый код.
3. Замените метод `get_statistics` на версию с подсчетом статистики по авторам (см. Шаг 9 в задании).

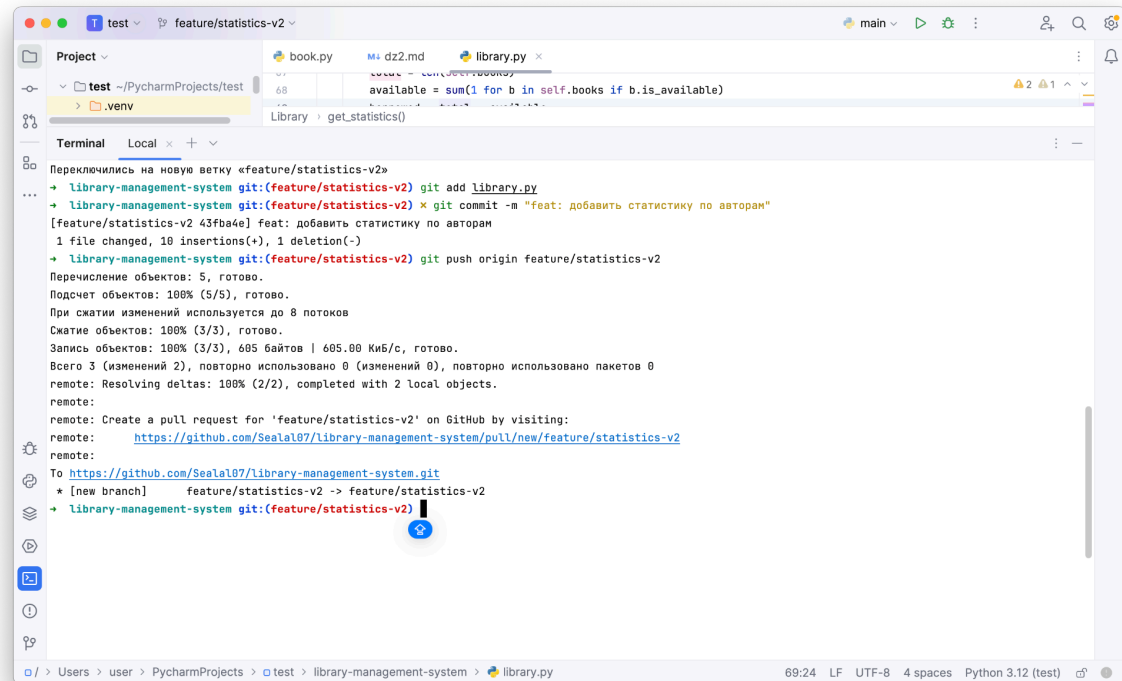


4. Отправьте код в облако:

```
git add library.py
```

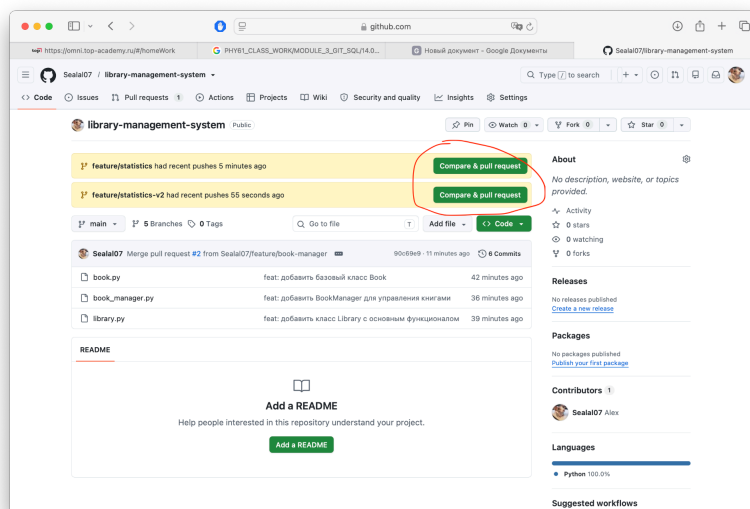
```
git commit -m "feat: добавить статистику по авторам"
```

```
git push origin feature/statistics-v2
```

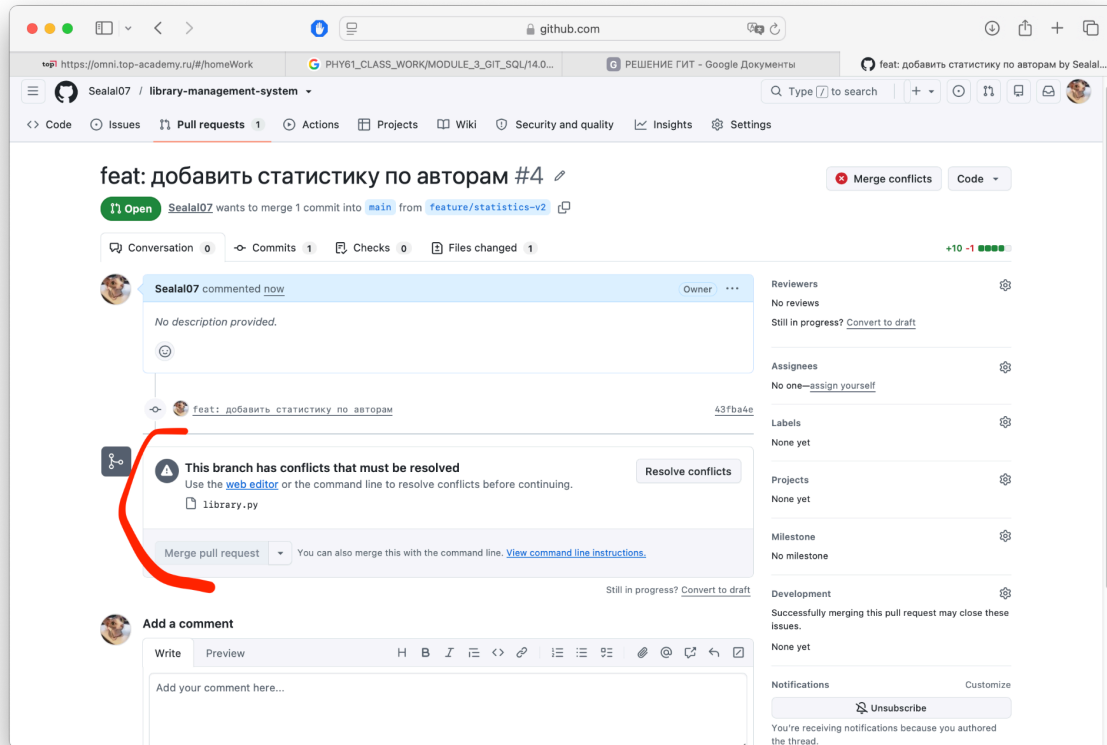


Шаг 10. Провокация конфликта и его решение

1. Зайдите на GitHub и создайте PR для ветки **feature/statistics**. Сделайте ему **Merge** в ветку **main**.



2. Теперь создайте PR для ветки `feature/statistics-v2`. GitHub покажет предупреждение: **"This branch has conflicts that must be resolved"** (невозможно объединить автоматически из-за конфликта в `library.py`).



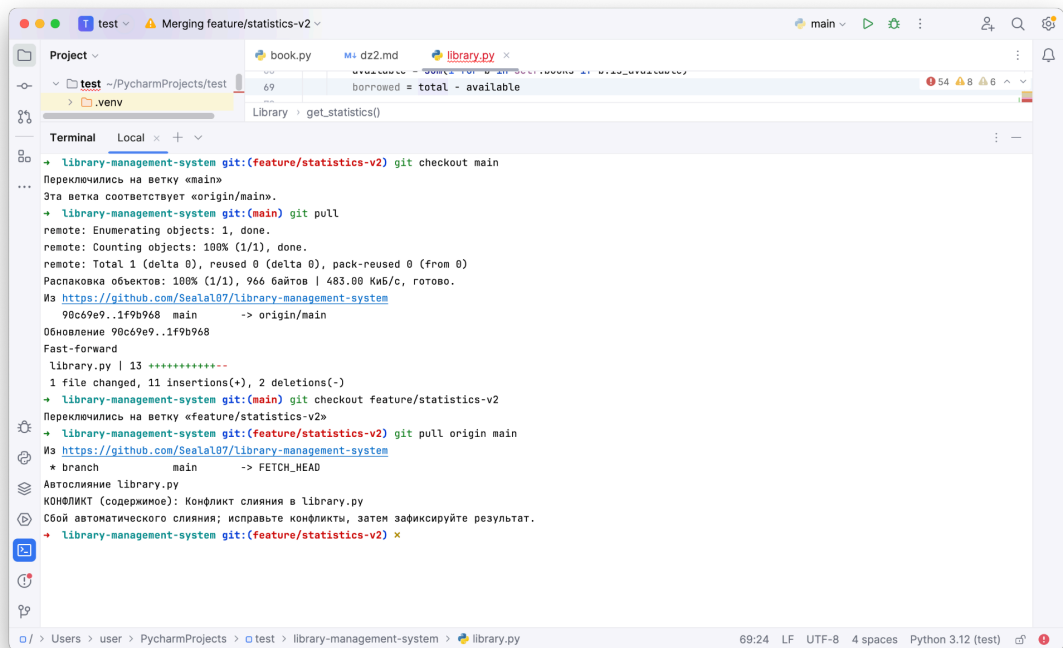
3. Разрешаем конфликт локально. Переходим в терминал:

```
git checkout main
git pull
```

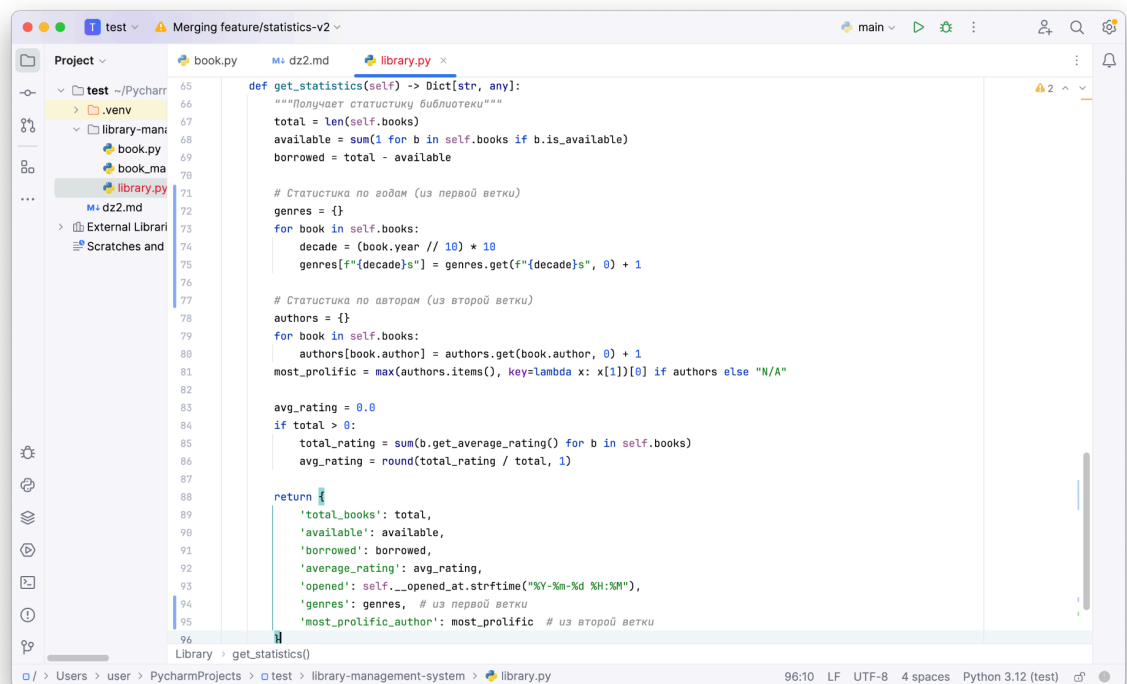
4. Затем переключаемся на вторую ветку и пытаемся влить в неё обновленный `main`:

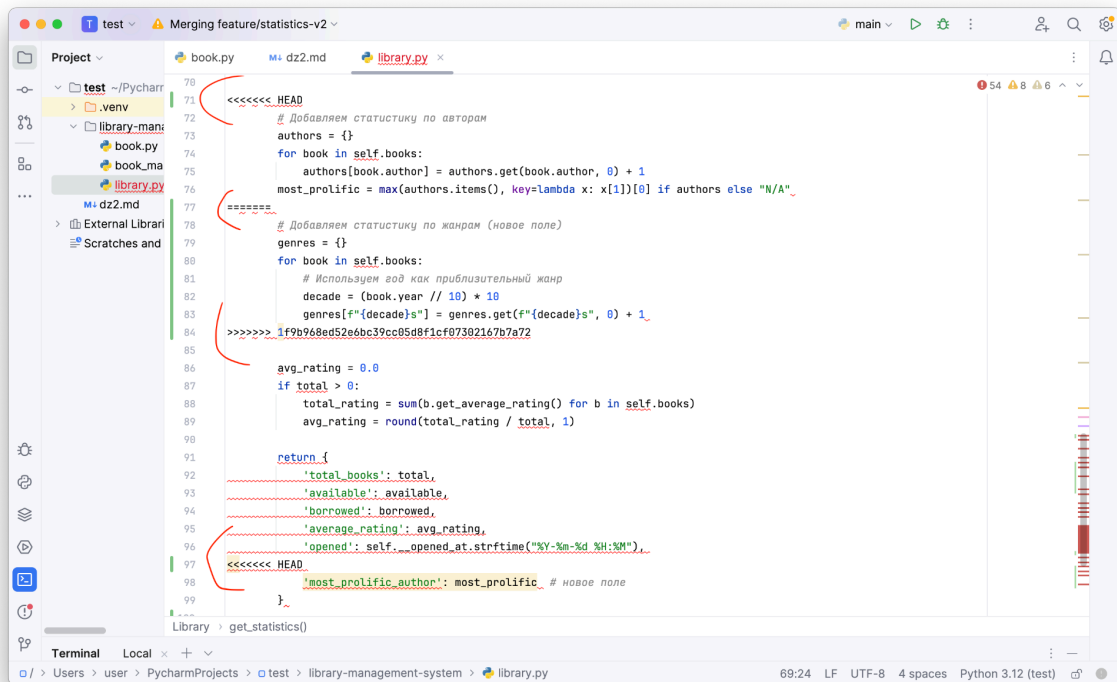
```
git checkout feature/statistics-v2
git pull origin main
```

5. Консоль напишет: `CONFLICT (content): Merge conflict in library.py`.



4. Разрешаем конфликт вручную: удалите маркеры Git и объедините код так, чтобы метод `get_statistics` считал и года, и авторов одновременно (итоговый код из Шага 10 задания).



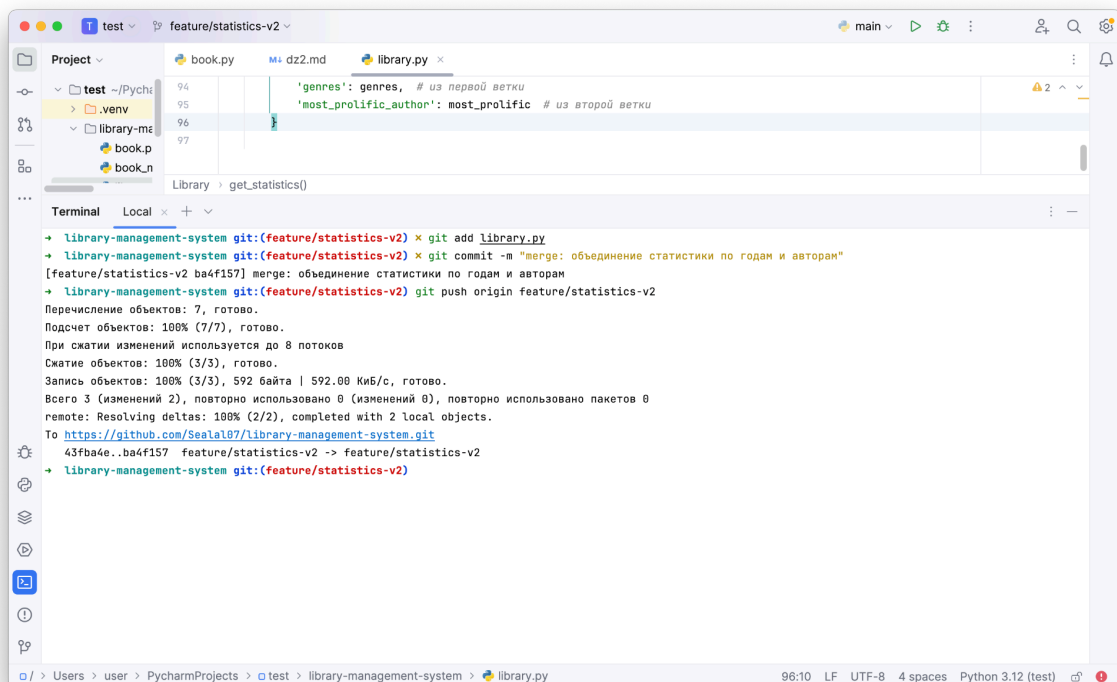


5. Сохраните файл и завершите слияние:

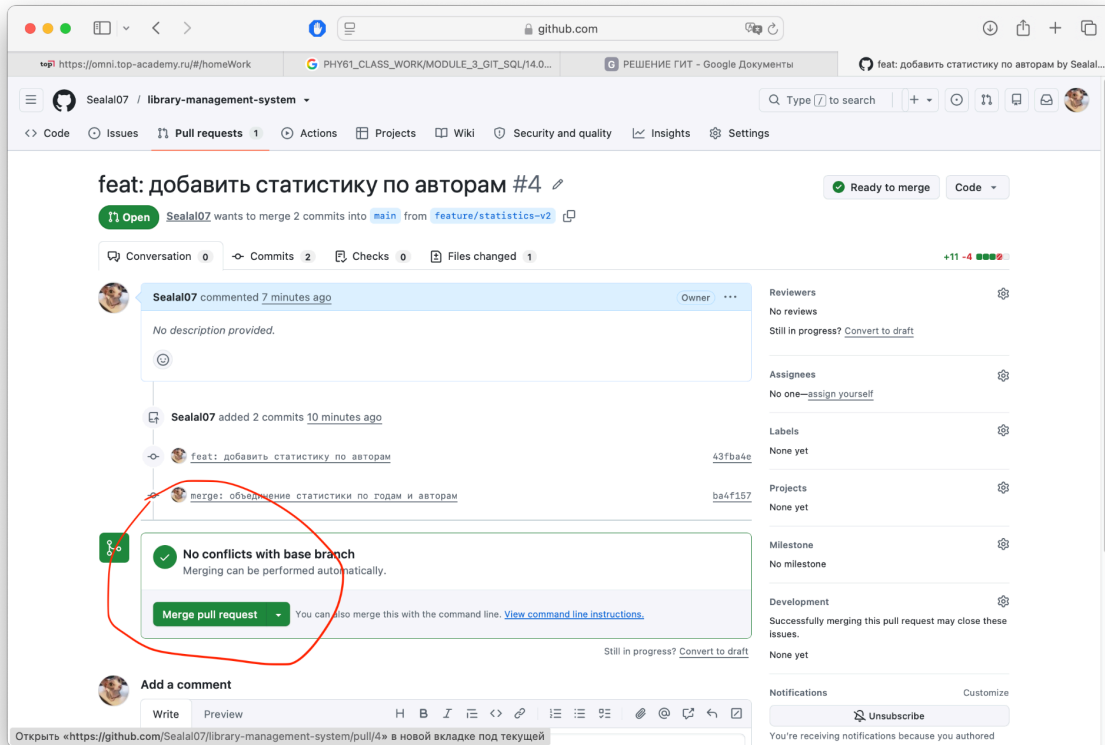
```
git add library.py
```

```
git commit -m "merge: объединение статистики по годам и авторам"
```

```
git push origin feature/statistics-v2
```



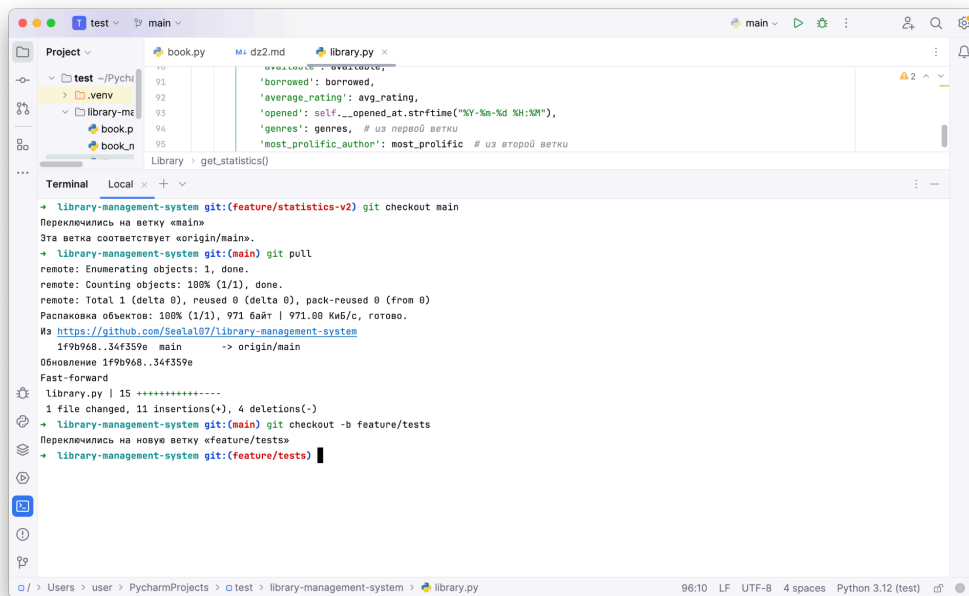
- Вернитесь на GitHub в PR ветки `feature/statistics-v2`. Теперь кнопка **Merge** стала зеленой и доступной. Нажмите её!



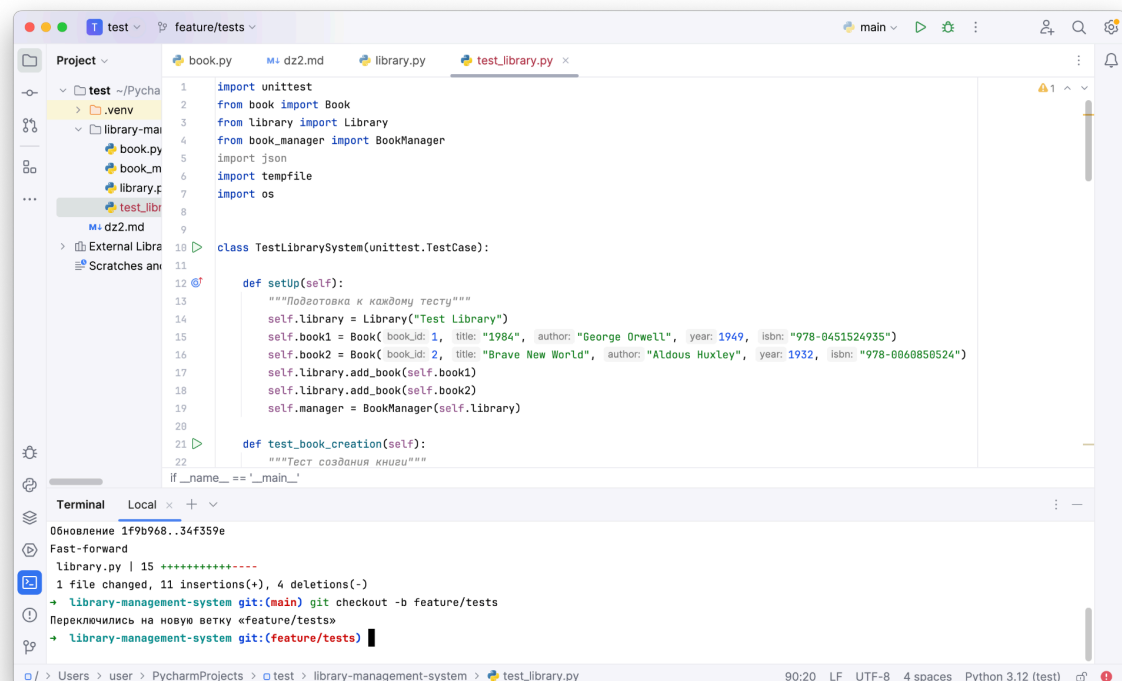
Шаг 11. Написание тестов

- Создаем новую ветку для тестов:

```
git checkout main
git pull
git checkout -b feature/tests
```

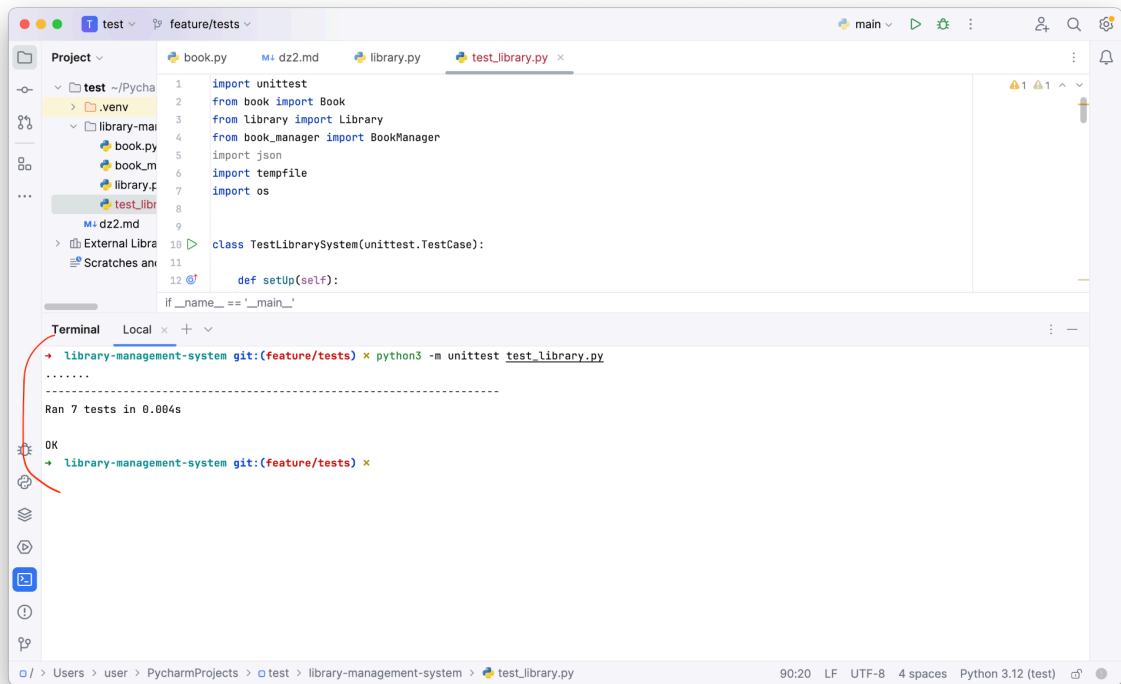



2. Создайте файл **test_library.py** и вставьте туда код тестов из задания.



3. Запустите тесты локально, чтобы убедиться, что всё работает:

python -m unittest test_library.py

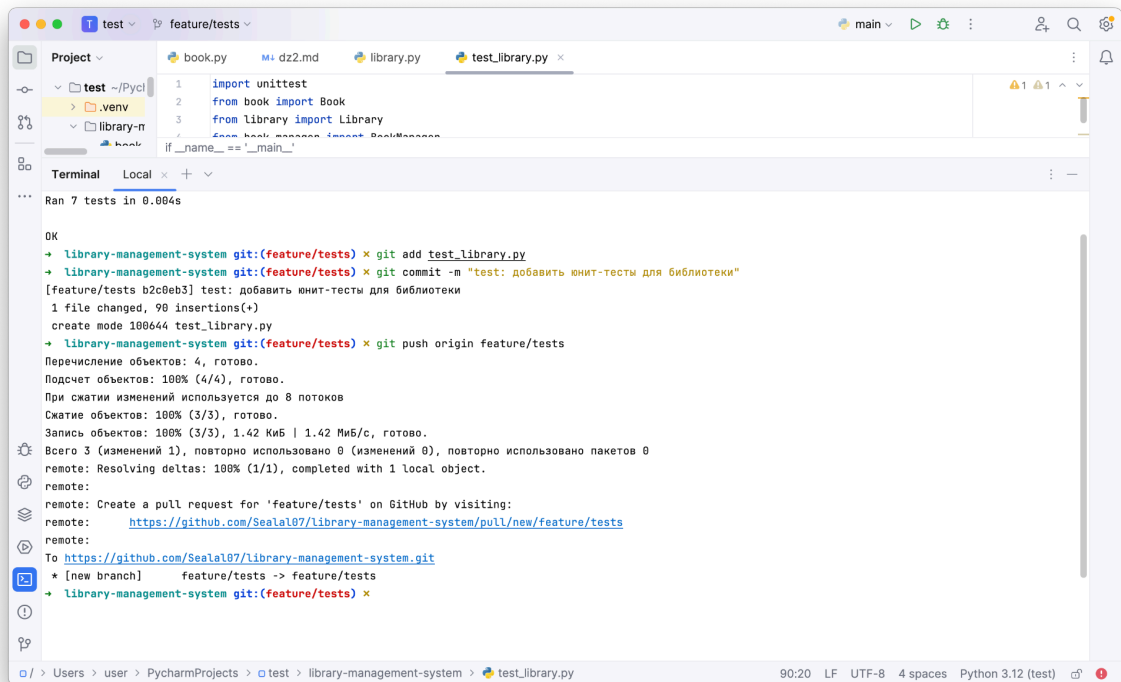


4. Отправляем тесты на GitHub:

```

git add test_library.py
git commit -m "test: добавить юнит-тесты для библиотеки"
git push origin feature/tests

```



Шаг 12. Финальный Merge и актуализация локального репозитория

1. Зайдите на GitHub, создайте PR для ветки `feature/tests` и сделайте ему **Merge**.
2. Обновите локальный репозиторий на компьютере, чтобы ветка `main` содержала абсолютно весь готовый код:

`git checkout main`

`git pull origin main`

```
...
Запись объектов: 100% (3/3), 1.42 КиБ | 1.42 МБ/с, готово.
Всего 3 (изменений 1), повторно использовано 0 (изменений 0), повторно использовано пакетов 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/tests' on GitHub by visiting:
remote:   https://github.com/Seala87/library-management-system/pull/new/feature/tests
remote:
To https://github.com/Seala87/library-management-system.git
* [new branch]   feature/tests -> feature/tests
+ library-management-system git:(feature/tests) * git checkout main
Переключились на ветку «main»
Эта ветка соответствует «origin/main».
+ library-management-system git:(main) * git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Распаковка объектов: 100% (1/1), 969 байтов | 969.00 КиБ/с, готово.
Из https://github.com/Seala87/library-management-system
* branch      main      -> FETCH_HEAD
   34f359e..c6ad469  main    -> origin/main
Обновление 34f359e..c6ad469
Fast-forward
 test_library.py | 90 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 90 insertions(+)
 create mode 100644 test_library.py
+ library-management-system git:(main) *
```